```
from prophet import Prophet
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.seasonal import seasonal_decompose
import random

from sklearn.metrics import mean_absolute_percentage_error
```

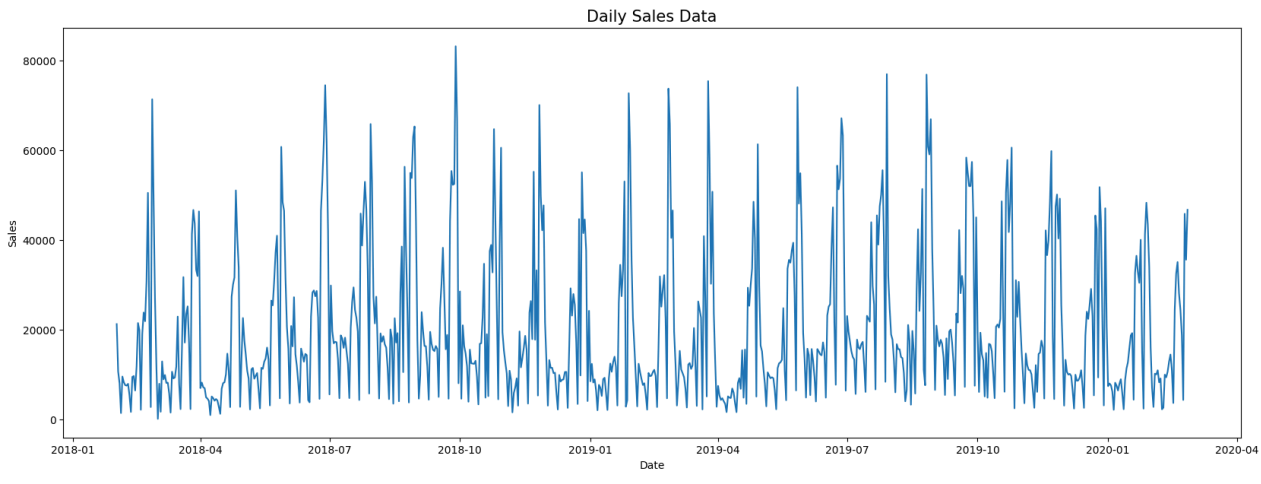
```
df = pd.read_excel(f"Groceries_Sales_data.xlsx",index_col=0)
```

```
df.head()
```

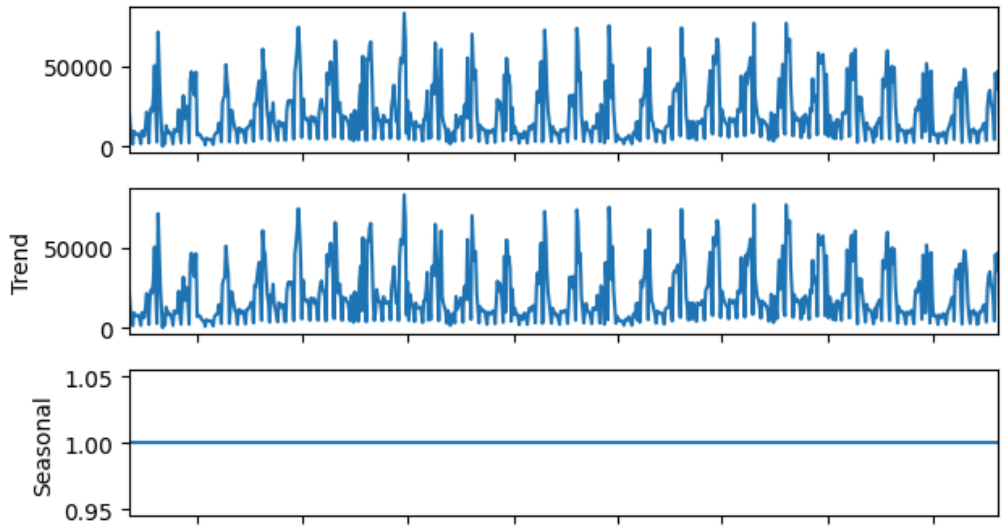
Date	Sales
2018-02-01	21199.0
2018-02-02	10634.0
2018-02-03	7966.0
2018-02-04	1353.0
2018-02-05	9497.0

Next steps: Generate code with df | View recommended plots | New interactive sheet

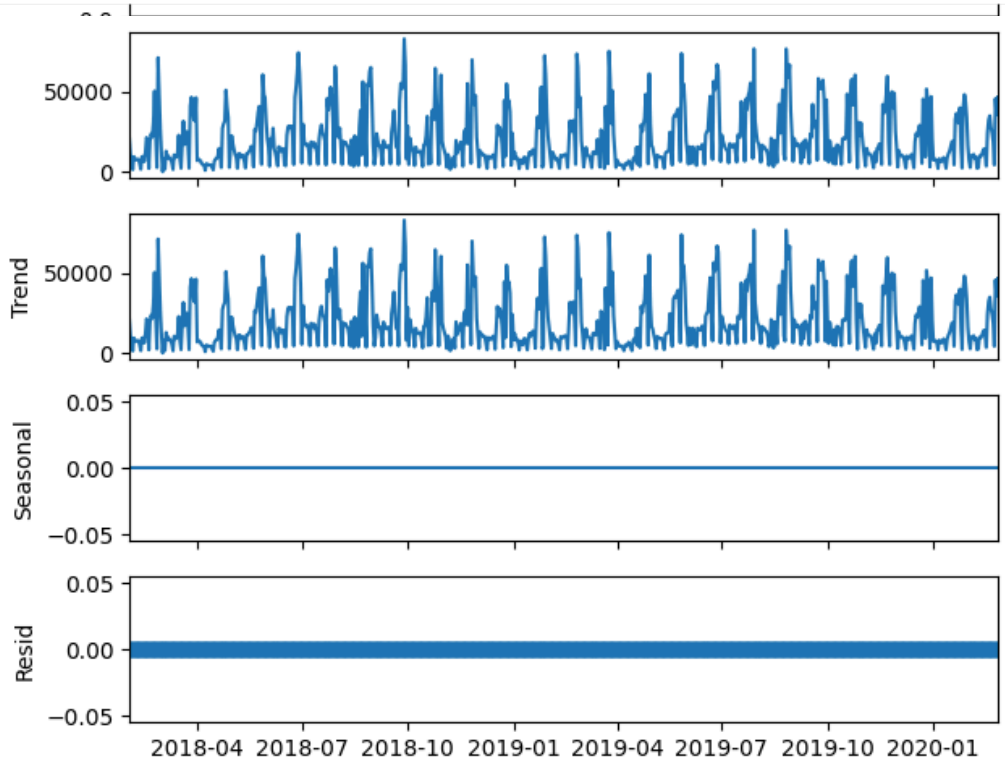
```
fig, ax = plt.subplots(figsize=(20,7))
a = sns.lineplot(x="Date", y="Sales", data=df)
a.set_title("Daily Sales Data",fontsize=15)
plt.show()
```




```
result_2 = seasonal_decompose(df, model='multiplicative', period=1)
result_2.plot()
plt.show()
```




```
result_3 = seasonal_decompose(df, model='additive', period=1)
result_3.plot()
plt.show()
```




```
Start coding or generate with AI.
```

```
df.index = pd.to_datetime(df.index)
```

```
df
```

Date	Sales
2018-02-01	21199.0
2018-02-02	10634.0
2018-02-03	7966.0
2018-02-04	1353.0
2018-02-05	9497.0
...	...
2020-02-22	18723.1

```
y = df["Sales"]
y.name = "Sales"
```

```
seasonal_df = y.to_frame()
seasonal_df
```

756 rows × 1 columns

Next steps: Generate code with df | View recommended plots | New interactive sheet

Date	Sales
2018-02-01	21199.0
2018-02-02	10634.0
2018-02-03	7966.0
2018-02-04	1353.0
2018-02-05	9497.0
...	...
2020-02-22	18723.1
2020-02-23	4274.9
2020-02-24	45805.7
2020-02-25	35566.3
2020-02-26	46703.0

756 rows × 1 columns

Next steps: Generate code with seasonal_df | View recommended plots | New interactive sheet

```
seasonal_df["trend"] = seasonal_df["Sales"].rolling(window=7, center=True).mean()
seasonal_df.head(10)
```

Date	Sales	trend
2018-02-01	21199.0	NaN
2018-02-02	10634.0	NaN

```
seasonal_df["detrended"] = seasonal_df["Sales"] - seasonal_df["trend"]
seasonal_df.head(10)
```

Date	Sales	trend	detrended
2018-02-01	21199.0	NaN	NaN
2018-02-02	10634.0	NaN	NaN
2018-02-03	7966.0	NaN	NaN
2018-02-04	1353.0	9491.000000	-8138.000000
2018-02-05	9497.0	7529.857143	1967.142857
2018-02-06	8207.0	7136.142857	1070.857143
2018-02-07	7581.0	6757.285714	823.714286
2018-02-08	7471.0	6793.285714	677.714286
2018-02-09	7878.0	6782.142857	1095.857143
2018-02-10	5314.0	6982.571429	-1668.571429

Next iew recommended plots New interactive sheet

Next steps: Generate code with seasonal_df View recommended plots New interactive sheet

```
seasonal_df.index = pd.to_datetime(seasonal_df.index)
seasonal_df["month"] = seasonal_df.index.month
seasonal_df["seasonality"] = seasonal_df.groupby("month")["detrended"].transform("mean")
seasonal_df.head(15)
```

Date	Sales	trend	detrended	month	seasonality
2018-02-01	21199.0	NaN	NaN	2	-112.66203

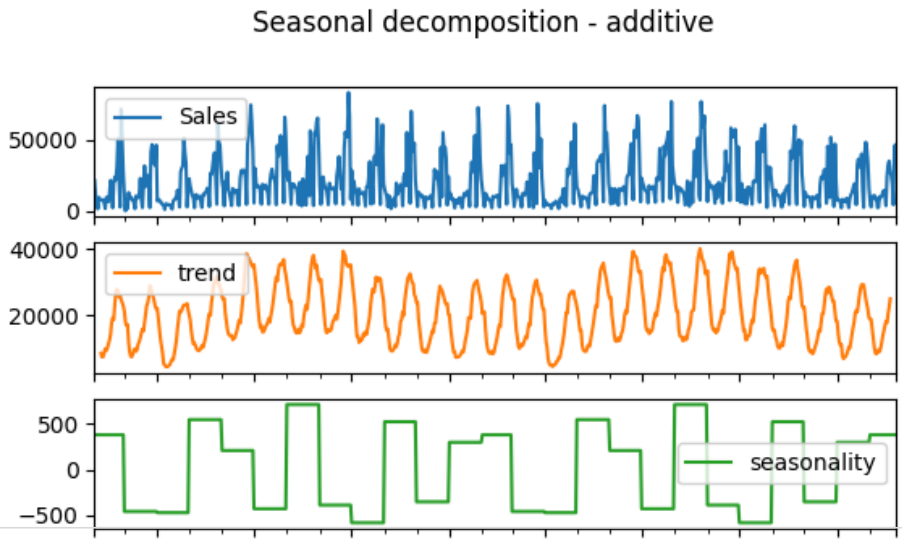
```
seasonal_df = y.to_frame()

# calculate the trend component
seasonal_df["trend"] = seasonal_df["Sales"].rolling(window=13, center=True).mean()

# detrend the series
seasonal_df["detrended"] = seasonal_df["Sales"] - seasonal_df["trend"]

# calculate the seasonal component
seasonal_df.index = pd.to_datetime(seasonal_df.index)
seasonal_df["month"] = seasonal_df.index.month
seasonal_df["seasonality"] = seasonal_df.groupby("month")["detrended"].transform("mean")

# get the residuals
seasonal_df["resid"] = seasonal_df["detrended"] - seasonal_df["seasonality"]

# display the DF
seasonal_df.head(15)
```

Date	Sales	trend	detrended	month	seasonality	resid
2018-02-01	21199.0	NaN	NaN	2	381.152198	NaN
2018-02-02	10634.0	NaN	NaN	2	381.152198	NaN
2018-02-03	7966.0	NaN	NaN	2	381.152198	NaN
2018-02-04	1353.0	NaN	NaN	2	381.152198	NaN
2018-02-05	9497.0	NaN	NaN	2	381.152198	NaN
2018-02-06	8207.0	NaN	NaN	2	381.152198	NaN
2018-02-07	7581.0	8287.230769	-706.230769	2	381.152198	-1087.382967
2018-02-08	7471.0	7147.930769	323.069231	2	381.152198	-58.082967
2018-02-09	7878.0	7237.546154	640.453846	2	381.152198	259.301648
2018-02-10	5314.0	8273.784615	-2959.784615	2	381.152198	-3340.936813
2018-02-11	1605.0	9693.061538	-8088.061538	2	381.152198	-8469.213736
2018-02-12	9419.0	9123.369231	295.630769	2	381.152198	-85.521429
2018-02-13	9610.0	9949.884615	-339.884615	2	381.152198	-721.036813
2018-02-14	6388.1	11195.930769	-4807.830769	2	381.152198	-5188.982967
2018-02-15	11799.0	12301.600000	-502.600000	2	381.152198	-883.752198

Next steps: Generate code with seasonal_df | View recommended plots | New interactive sheet

```
seasonal_df.loc[:, ["Sales", "trend", "seasonality", "resid"]].plot(subplots=True, title="Seasonal dec
```

```
array([<Axes: xlabel='Date'>, <Axes: xlabel='Date'>,
       <Axes: xlabel='Date'>, <Axes: xlabel='Date'>], dtype=object)
```




```
df=df.reset_index(['Date'])
```

```
df
```

	Date	Sales
0	2018-02-01	21199.0
1	2018-02-02	10634.0
2	2018-02-03	7966.0
3	2018-02-04	1353.0
4	2018-02-05	9497.0
...	...	...
751	2020-02-22	18723.1
752	2020-02-23	4274.9
753	2020-02-24	45805.7
754	2020-02-25	35566.3
755	2020-02-26	46703.0

756 rows × 2 columns

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
# Extract features
df_copy = df.copy()

df_copy['date'] = df['Date']
df_copy['month'] = df_copy['date'].dt.strftime('%B')
df_copy['year'] = df_copy['date'].dt.strftime('%Y')
df_copy['dayofweek'] = df_copy['date'].dt.strftime('%A')
df_copy['quarter'] = df_copy['date'].dt.quarter
df_copy['dayofyear'] = df_copy['date'].dt.dayofyear
df_copy['dayofmonth'] = df_copy['date'].dt.day
df_copy['weekofyear'] = df_copy['date'].dt.isocalendar().week
```

```
df_copy.columns
```

```
Index(['Date', 'Sales', 'date', 'month', 'year', 'dayofweek', 'quarter',
       'dayofyear', 'dayofmonth', 'weekofyear'],
```

```
        dtype='object')
```

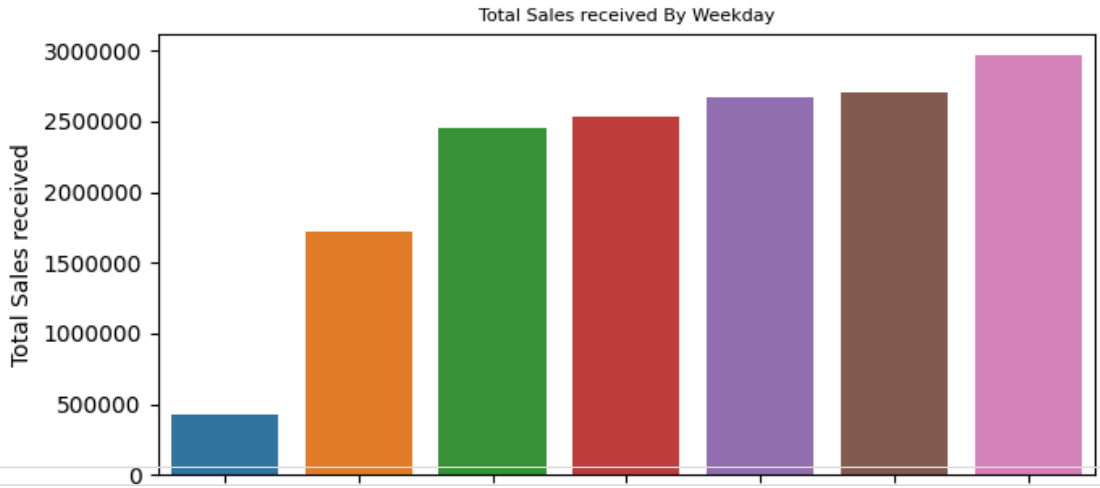
```
X = df_copy[['dayofweek','quarter','month','year',
           'dayofyear','dayofmonth','weekofyear']]
y = df['Sales']
```

```
df_new = pd.concat([X, y], axis=1)
df_new.head()
```

	dayofweek	quarter	month	year	dayofyear	dayofmonth	weekofyear	Sales
0	Thursday	1	February	2018	32	1	5	21199.0
1	Friday	1	February	2018	33	2	5	10634.0
2	Saturday	1	February	2018	34	3	5	7966.0
3	Sunday	1	February	2018	35	4	5	1353.0
4	Monday	1	February	2018	36	5	6	9497.0

Next steps: Generate code with df_new | View recommended plots | New interactive sheet

```
# fig,(ax1,ax2,ax3,ax4)= plt.subplots(nrows=4)
fig,(ax1,ax2)= plt.subplots(nrows=2)
fig.set_size_inches(7,7)

week_day_Aggregated = pd.DataFrame(df_new.groupby("dayofweek")["Sales"].sum()).reset_index().sort_valu
sns.barplot(data=week_day_Aggregated,x="dayofweek",y="Sales",hue = 'dayofweek',ax=ax1,dodge=False)
ax1.set(xlabel='dayofweek', ylabel='Total Sales received')
ax1.xaxis.label.set_size(8)
ax1.set_title("Total Sales received By Weekday",fontsize=8)
ax1.ticklabel_format(style='plain',axis='y')



yearAggregated = pd.DataFrame(df_new.groupby("year")["Sales"].sum()).reset_index()
sns.barplot(data=yearAggregated,x="year",y="Sales",hue='year',ax=ax2)
ax2.set(xlabel='year', ylabel='Total Sales received')
ax2.xaxis.label.set_size(8)
ax2.set_title("Total Sales received By year",fontsize=8)
ax2.ticklabel_format(style='plain',axis='y')

fig.tight_layout()
```



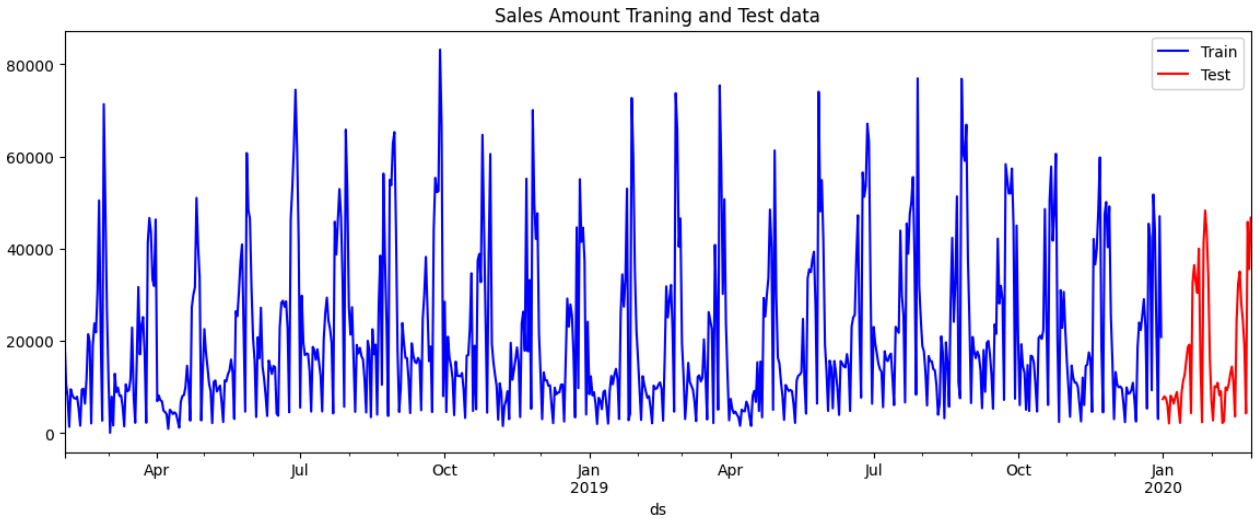

```
# Train test split
```

```
df=df.rename(columns={'Date':'ds','Sales':'y'})
df.tail()
```

	ds	y
751	2020-02-22	18723.1
752	2020-02-23	4274.9
753	2020-02-24	45805.7
754	2020-02-25	35566.3
755	2020-02-26	46703.0

```
end_date = '2019-12-31'
X_tr = df.loc[df['ds'] <= end_date]
X_tst = df.loc[df['ds'] > end_date]
```

```
pd.plotting.register_matplotlib_converters()
f, ax = plt.subplots(figsize=(14,5))
X_tr.plot(kind='line', x='ds', y='y', color='blue', label='Train', ax=ax)
X_tst.plot(kind='line', x='ds', y='y', color='red', label='Test', ax=ax)
plt.title('Sales Amount Traning and Test data')
plt.show()
```




```
# Modelling
```

```
model =Prophet()
model.fit(X_tr)
```

```
INFO:prophet:Disabling yearly seasonality. Run prophet with yearly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/ukzovhhv.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/eslqmxhp.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mod
17:09:42 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
17:09:42 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
<prophet.forecaster.Prophet at 0x79c95dc20230>
```

```
len(X_tst)
```

```
57
```

```
X_tst
```

Next steps: Generate code with X_tst | View recommended plots | New interactive sheet

```
future = model.make_future_dataframe(periods=57, freq='D')
forecast = model.predict(future)
forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].head()
```

	dsds	yhat	yhat_lower	yhat_upper
0	2018-02-01	22918.473941	3872.005353	41503.482212
1	2018-02-02	21609.554668	2519.790011	42737.351208
2	2018-02-03	14076.387315	-6155.830286	33008.425861
3	2018-02-04	16696.213765	-18174.615354	20356.617122
4	2018-02-05	25535.083176	5574.041480	45782.812922

```
future_2 = model.make_future_dataframe(periods=60)
forecast_2 = model.predict(future_2)
forecast_2[['ds', 'yhat']].tail(5)
```

	ds	yhat
754	2020-02-25	27945.349608
755	2020-02-26	25546.363061
756	2020-02-27	28114.586689
757	2020-02-28	26805.667405
758	2020-02-29	19272.500042

```
X_tst_forecast = model.predict(X_tst)
X_tst_forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(7)
```

	ds	yhat	yhat_lower	yhat_upper
50	2020-02-20	28066.474534	9679.004539	48549.416446
51	2020-02-21	26757.555250	6043.868983	45610.964814
52	2020-02-22	19224.387887	-30.779434	39369.707666
53	2020-02-23	6817.012325	-11933.775066	26901.925420
54	2020-02-24	30683.083725	10835.183229	48601.625482
55	2020-02-25	27945.349608	8591.968694	46975.719037
56	2020-02-26	25546.363061	4751.243183	45714.731542

```
X_tst_forecast
```

726	2020-01-28	48276.2
727	2020-01-29	43421.1
728	2020-01-30	33991.7
729	2020-01-31	16010.9
730	2020-02-01	6986.7
731	2020-02-02	2718.4
732	2020-02-03	10131.1
733	2020-02-04	9970.3
734	2020-02-05	10905.6
735	2020-02-06	8210.8
736	2020-02-07	9080.8
737	2020-02-08	2191.7
738	2020-02-09	2559.2

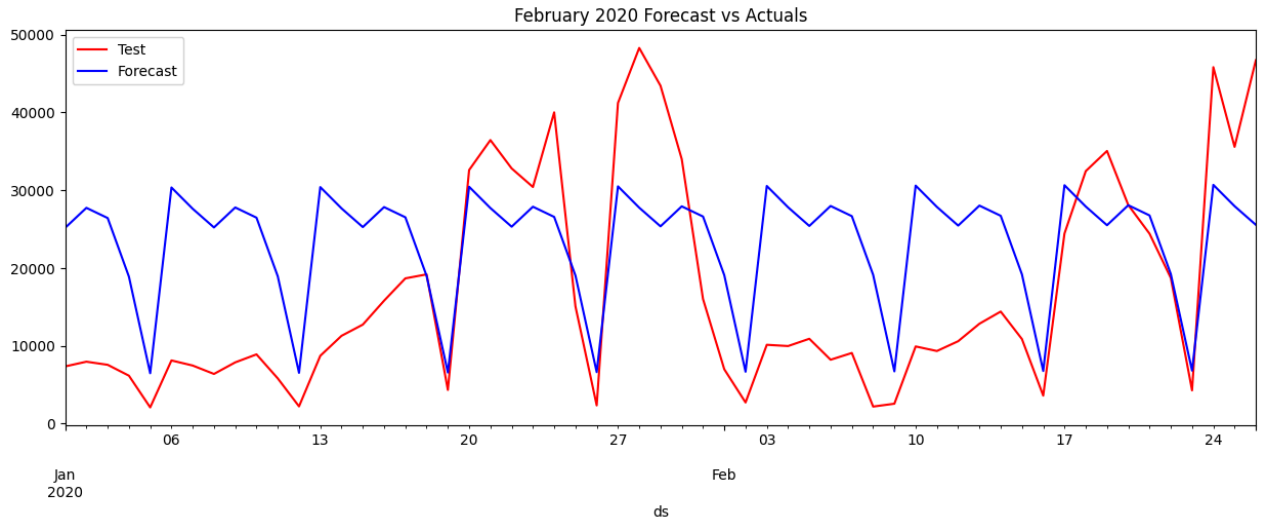
739	2020-02-10	9920.1
740	2020-02-11	9332.1
741	2020-02-12	10593.3
742	2020-02-13	12838.9
743	2020-02-14	14402.9
744	2020-02-15	10866.3
745	2020-02-16	3598.1
746	2020-02-17	24425.7
747	2020-02-18	32450.5
748	2020-02-19	35041.9
749	2020-02-20	28088.8
750	2020-02-21	24412.5
751	2020-02-22	18723.1
752	2020-02-23	4274.9
753	2020-02-24	45805.7
754	2020-02-25	35566.3
755	2020-02-26	46703.0

	ds	trend	yhat_lower	yhat_upper	trend_lower	trend_upper	additive_terms	addit:
0	2020-01-01	23212.897455	4557.077982	43771.282072	23212.897455	23212.897455	1948.568368	
1	2020-01-02	23219.770620	8665.117984	47651.918904	23219.770620	23219.770620	4509.918830	
2	2020-01-03	23226.643785	7979.248545	45239.105963	23226.643785	23226.643785	3194.126381	
3	2020-01-04	23233.516950	-929.092719	38174.940113	23233.516950	23233.516950	-4345.914147	
4	2020-01-05	23240.390115	-13600.919238	25494.804482	23240.390115	23240.390115	-16760.162873	
5	2020-01-06	23247.263280	12677.865890	51125.731815	23247.263280	23247.263280	7099.035362	
6	2020-01-07	23254.136445	7997.766806	46600.887408	23254.136444	23254.136446	4354.428080	
7	2020-01-08	23261.009610	5272.398960	43631.376742	23261.009608	23261.009611	1948.568368	
8	2020-01-09	23267.882775	7935.418547	47536.250067	23267.882772	23267.882777	4509.918830	
9	2020-01-10	23274.755940	6246.675872	45687.918780	23274.755935	23274.755943	3194.126381	
10	2020-01-11	23281.629105	-1398.839117	38508.130523	23281.629098	23281.629109	-4345.914147	
11	2020-01-12	23288.502270	-12991.056401	25089.818953	23288.502262	23288.502275	-16760.162873	
12	2020-01-13	23295.375435	11987.928708	50146.517214	23295.375425	23295.375442	7099.035362	
13	2020-01-14	23302.248600	8699.567101	47505.907817	23302.248588	23302.248608	4354.428079	
14	2020-01-15	23309.121765	5276.292852	44845.868893	23309.121750	23309.121775	1948.568368	
15	2020-01-16	23315.994930	9797.870573	46708.266194	23315.994913	23315.994941	4509.918830	
16	2020-01-17	23322.868095	7204.506033	46166.203061	23322.868075	23322.868108	3194.126381	
17	2020-01-18	23329.741260	217.861516	38555.288859	23329.741237	23329.741275	-4345.914147	
18	2020-01-19	23336.614425	-12252.889399	24219.405038	23336.614400	23336.614442	-16760.162873	
19	2020-01-20	23343.487590	10040.280900	50733.719883	23343.487563	23343.487610	7099.035362	
20	2020-01-21	23350.360755	9029.962460	47833.695686	23350.360726	23350.360777	4354.428079	
21	2020-01-22	23357.233920	5621.290057	44568.495502	23357.233888	23357.233945	1948.568368	
22	2020-01-23	23364.107085	9135.904787	46807.841752	23364.107050	23364.107113	4509.918830	
23	2020-01-24	23370.980250	8487.547392	45476.612405	23370.980212	23370.980282	3194.126381	

Next steps: Generate code with X_tst_forecast | View recommended plots | New interactive sheet

```
f, ax = plt.subplots(figsize=(14,5))
f.set_figheight(5)
f.set_figwidth(15)
X_tst.plot(kind='line',x='ds', y='y', color='red', label='Test', ax=ax)
```

```
X_tst_forecast.plot(kind='line',x='ds',y='yhat', color='blue',label='Forecast', ax=ax)
plt.title('February 2020 Forecast vs Actuals')
plt.show()
```



34	2020-02-04	23446.585064	7999.403554	46381.668055	23446.584983	23446.585132	4354.428079

```
mape = mean_absolute_percentage_error(X_tst['y'],X_tst_forecast['yhat'])
print("MAPE",mape)
```

```
MAPE 1.2952508075863325
```

36	2020-02-06	23460.331394	8485.734091	46715.509853	23460.331303	23460.331467	4509.918830

```
# Add Holidays import holidays which is a different libraries
# Handdling holidays
```

```
import holidays
```

```
india_holidays = holidays.India(years = 2018)

holiday_india_df = pd.DataFrame([])

for date, name in sorted(india_holidays.items()):
    print(date, name)
    holiday_india_df = pd.concat([holiday_india_df, pd.DataFrame({'ds': date, 'holiday': name}, index=
```

42	2020-02-12	23501.570384	6465.871290	44229.088931	23501.570259	23501.570477	1948.568368

```
2018-01-26 Republic Day
2018-02-13 Maha Shivaratri
2018-03-29 Mahavir Jayanti
2018-03-30 Good Friday
2018-04-30 Buddha Purnima
2018-06-16 Eid al-Fitr
2018-08-15 Independence Day
2018-08-22 Eid al-Adha
2018-08-23 Janmashtami
2018-09-21 Ashura
2018-10-02 Gandhi Jayanti
2018-10-19 Dussehra
2018-11-07 Diwali
2018-11-21 Prophet's Birthday
2018-11-23 Guru Nanak Jayanti
2018-12-25 Christmas
```

43	2020-02-13	23508.443549	8167.829535	47145.508575	23508.443419	23508.443648	4509.918830
44	2020-02-14	23515.316714	6177.724284	45810.976604	23515.316581	23515.316816	3194.126381
45	2020-02-15	23522.189879	-2670.136000	38095.977097	23522.189744	23522.189983	-4345.914147
46	2020-02-16	23529.063044	-12330.675851	26394.403123	23529.062904	23529.063151	-16760.162873
47	2020-02-17	23535.936209	1419.308341	50826.007367	23535.936067	23535.936322	7099.035362
48	2020-02-18	23542.809374	7703.033282	47249.931085	23542.809226	23542.809492	4354.428079

```
holiday_india_df
```

	02-19						
50	2020-02-20	23556.555704	9679.004539	48549.416446	23556.555545	23556.555833	4509.918830
51	2020-02-21	23563.428869	6043.868983	45610.964814	23563.428706	23563.429001	3194.126381
	2020-						

	ds	holiday
0	2018-01-26	Republic Day
1	2018-02-13	Maha Shivaratri
2	2018-03-29	Mahavir Jayanti
3	2018-03-30	Good Friday
4	2018-04-30	Buddha Purnima
5	2018-06-16	Eid al-Fitr
6	2018-08-15	Independence Day
7	2018-08-22	Eid al-Adha
8	2018-09-03	Janmashtami
9	2018-09-21	Ashura
10	2018-10-02	Gandhi Jayanti
11	2018-10-19	Dussehra
12	2018-11-07	Diwali
13	2018-11-21	Prophet's Birthday

```
india_holidays_mh = holidays.India(years = 2018,subdiv='MH')

holiday_india_mh = pd.DataFrame([])

for date, name in sorted(india_holidays_mh.items()):
    print(date, name)
    holiday_india_mh = pd.concat([holiday_india_mh, pd.DataFrame({'ds': date, 'holiday': name}, index=
```

```
2018-01-26 Republic Day
2018-02-13 Maha Shivaratri
2018-02-19 Chhatrapati Shivaji Maharaj Jayanti
2018-03-18 Gudi Padwa
2018-03-29 Mahavir Jayanti
2018-03-30 Good Friday
2018-04-14 Dr. B. R. Ambedkar's Jayanti
2018-04-30 Buddha Purnima
2018-05-01 Maharashtra Day
2018-06-16 Eid al-Fitr
2018-08-15 Independence Day
2018-08-22 Eid al-Adha
2018-09-03 Janmashtami
2018-09-21 Ashura
2018-10-02 Gandhi Jayanti
2018-10-19 Dussehra
2018-11-07 Diwali
2018-11-21 Prophet's Birthday
2018-11-23 Guru Nanak Jayanti
2018-12-25 Christmas
```

```
holiday_india_mh
```

	ds	holiday
0	2018-01-26	Republic Day
1	2018-02-13	Maha Shivaratri
2	2018-02-19	Chhatrapati Shivaji Maharaj Jayanti
3	2018-03-18	Gudi Padwa
4	2018-03-29	Mahavir Jayanti
5	2018-03-30	Good Friday
6	2018-04-14	Dr. B. R. Ambedkar's Jayanti
7	2018-04-30	Buddha Purnima
8	2018-05-01	Maharashtra Day
9	2018-06-16	Eid al-Fitr
10	2018-08-15	Independence Day
11	2018-08-22	Eid al-Adha
12	2018-09-03	Janmashtami
13	2018-09-21	Ashura
17	2018-11-21	Prophet's Birthday
18	2018-11-23	Guru Nanak Jayanti
19	2018-12-25	Christmas

```
holiday = pd.DataFrame([])
for date, name in sorted(holidays.UnitedStates(years=[2018,2019,2020]).items()):
    holiday = pd.concat([holiday, pd.DataFrame({'ds': date, 'holiday': "US-Holidays"}, index=[0])], i
holiday['ds'] = pd.to_datetime(holiday['ds'], format='%Y-%m-%d', errors='ignore')
```

```
/tmp/ipython-input-3775490065.py:4: FutureWarning: errors='ignore' is deprecated and will raise in a f
  holiday['ds'] = pd.to_datetime(holiday['ds'], format='%Y-%m-%d', errors='ignore')
```

```
holiday.tail(7)
```

Next steps: Generate code with holiday_india_mh View recommended plots New interactive sheet

	ds	holiday
25	2020-07-03	US-Holidays
26	2020-07-04	US-Holidays
27	2020-09-07	US-Holidays
28	2020-10-12	US-Holidays
29	2020-11-11	US-Holidays
30	2020-11-26	US-Holidays
31	2020-12-25	US-Holidays

```
holiday_2 = pd.DataFrame([])

for date, name in sorted(holidays.India(years=[2018,2019,2020]).items()):
    holiday_2 = pd.concat([holiday_2, pd.DataFrame({'ds': date, 'holiday': "India-Holidays"}, index=[
holiday_2['ds'] = pd.to_datetime(holiday_2['ds'], format='%Y-%m-%d', errors='ignore')
```

```
/tmp/ipython-input-2575076591.py:5: FutureWarning: errors='ignore' is deprecated and will raise in a f
  holiday_2['ds'] = pd.to_datetime(holiday_2['ds'], format='%Y-%m-%d', errors='ignore')
```

```
# if the freq does not match then we will extract some feature like month day and then match it with
# We can add holidays by using method we need to add country name as well
# it is to be done before fitting the model
```

```
m_2 = Prophet()
m_2.add_country_holidays(country_name='IN')
```

```
m_2.fit(df)
```

```
/usr/local/lib/python3.12/dist-packages/holidays/countries/india.py:182: Warning: Requested Holidays a
  warnings.warn(warning_msg, Warning)
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/5nktifse.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/avnwcvj6.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mod
17:15:07 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
17:15:07 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
<prophet.forecaster.Prophet at 0x79c956f8c3b0>
```

```
m_2.train_holiday_names
```

	0
0	Republic Day
1	Independence Day
2	Gandhi Jayanti
3	Buddha Purnima
4	Diwali
5	Janmashtami
6	Dussehra
7	Mahavir Jayanti
8	Maha Shivaratri
9	Guru Nanak Jayanti
10	Ashura
11	Prophet's Birthday
12	Eid al-Fitr
13	Eid al-Adha
14	Good Friday
15	Christmas

dtype: object

```
# fit with holidays
```

```
model_with_holidays = Prophet(holidays=holiday)
model_with_holidays.fit(X_tr)
```

```
INFO:prophet:Disabling yearly seasonality. Run prophet with yearly_seasonality=True to override this.
INFO:prophet:Disabling daily seasonality. Run prophet with daily_seasonality=True to override this.
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/vhrjur91.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/zk64khwv.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mod
17:15:17 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
17:15:17 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
<prophet.forecaster.Prophet at 0x79c956f8e240>
```

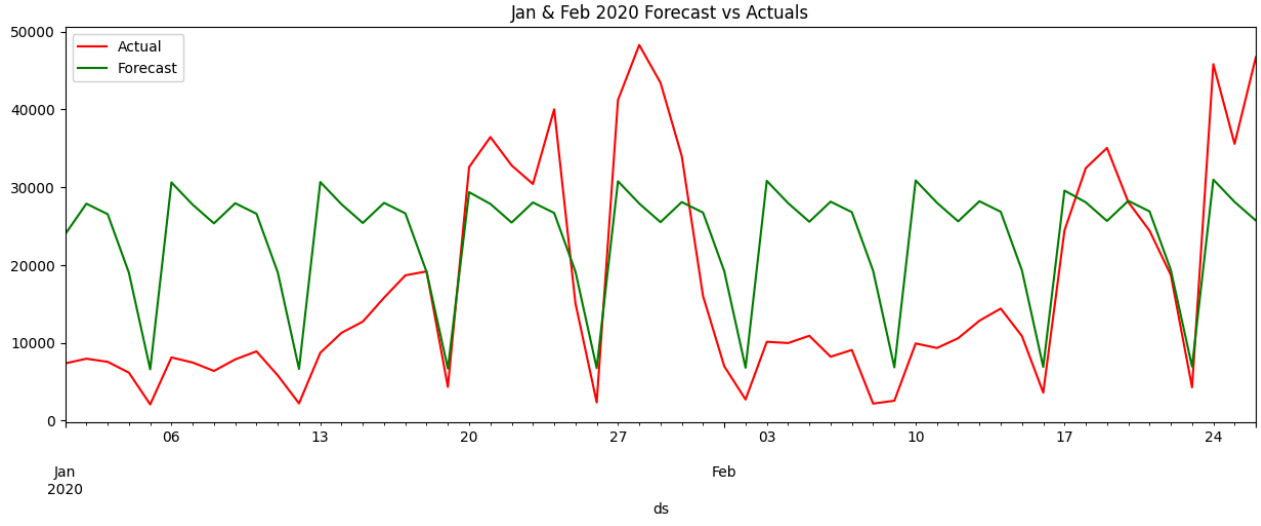
```
future_holiday = model_with_holidays.make_future_dataframe(periods=57, freq='D')
forecast_holiday = model_with_holidays.predict(future_holiday)
forecast_holiday[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(7)
```

	ds	yhat	yhat_lower	yhat_upper
749	2020-02-20	28244.407248	9003.761811	46001.289712
750	2020-02-21	26893.953958	8069.119051	46572.238611
751	2020-02-22	19356.400851	179.456809	37625.973267
752	2020-02-23	6955.481209	-11846.626649	26770.306085
753	2020-02-24	30969.646063	10672.838501	49309.284410
754	2020-02-25	28108.442406	9797.130526	47089.119202
755	2020-02-26	25710.999870	6112.670118	43856.291037

```
X_tst_forecast_holiday = model_with_holidays.predict(X_tst)
X_tst_forecast_holiday[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(7)
```

	ds	yhat	yhat_lower	yhat_upper
50	2020-02-20	28244.407248	9244.790734	48935.921974
51	2020-02-21	26893.953958	6791.866995	45329.763666
52	2020-02-22	19356.400851	1038.979224	37677.214270
53	2020-02-23	6955.481209	-10909.651237	24636.484836
54	2020-02-24	30969.646063	11179.964469	48307.933230
55	2020-02-25	28108.442406	9905.184598	45843.631887
56	2020-02-26	25710.999870	5993.639500	44267.069800

```
f, ax = plt.subplots(figsize=(14,5))
f.set_figheight(5)
f.set_figwidth(15)
X_tst.plot(kind='line',x='ds', y='y', color='red', label='Actual', ax=ax)
X_tst_forecast_holiday.plot(kind='line',x='ds',y='yhat', color='green',label='Forecast', ax=ax)
plt.title('Jan & Feb 2020 Forecast vs Actuals')
plt.show()
```



```
mape_holiday = mean_absolute_percentage_error(X_tst['y'],X_tst_forecast_holiday['yhat'])
print("MAPE",round(mape_holiday,4))
```

```
MAPE 1.3068
```

```
# Hyperparameter tunning
```

```
# n_changepoints is the number of change happen in the data. Prophet model detects them by its own. B

# changepoint_prior_scale to indicate how flexible the changepoints are allowed to be. In other words

# seasonality_mode There are 2 types model seasonality mode. Additive & multiplicaticative. By defaul

# holiday_prior_scale just like changepoint_prior_scale, holiday_prior_scale is used to smoothning th

# Seasonalities with fourier_order Prophet model, by default finds the seasonalities and adds the def
# seasonality_weekly / seasonality_weekly = True / Fl
```

```
from sklearn.model_selection import ParameterGrid
params_grid = {'seasonality_mode':('multiplicative','additive'),
               'changepoint_prior_scale':[0.1,0.2,0.3],
              'holidays_prior_scale':[0.1,0.2,0.3],
              'n_changepoints' : [100,150]}
grid = ParameterGrid(params_grid)
cnt = 0
for p in grid:
    cnt = cnt+1

print('Total Possible Models',cnt)
```

```
Total Possible Models 36
```

```
%%timeit
strt='2019-12-31'
end='2020-02-26'
model_parameters = pd.DataFrame(columns = ['MAPE','Parameters'])
for p in grid:
    test = pd.DataFrame()
    print(p)
    random.seed(0)
    train_model =Prophet(changepoint_prior_scale = p['changepoint_prior_scale'],
                         holidays_prior_scale = p['holidays_prior_scale'],
                         n_changepoints = p['n_changepoints'],
                         seasonality_mode = p['seasonality_mode'],
                         weekly_seasonality=True,
                         daily_seasonality = True,
                         yearly_seasonality = True,
                         holidays=holiday,
                         interval_width=0.95)
    train_model.add_country_holidays(country_name='US')
    train_model.fit(X_tr)
    train_forecast = train_model.make_future_dataframe(periods=57, freq='D',include_history = False)
    train_forecast = train_model.predict(train_forecast)
    test=train_forecast[['ds','yhat']]
    Actual = df[(df['ds']>strt) & (df['ds']<=end)]
    MAPE = mean_absolute_percentage_error(Actual['y'],abs(test['yhat']))
    print('Mean Absolute Percentage Error(MAPE)------------------------------------',MAPE)
    model_parameters = pd.concat([model_parameters, pd.DataFrame({'MAPE':MAPE,'Parameters':p}, index=[0
```

```
INFO:cmdstanpy:Chain [1] done processing
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/gs47vasu.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/nwz6gq1j.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mo
17:17:43 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
Mean Absolute Percentage Error(MAPE)----------------------------------- 1.0492091623916695
{'changepoint_prior_scale': 0.1, 'holidays_prior_scale': 0.1, 'n_changepoints': 150, 'seasonality_mo
17:17:43 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/jz65_us8.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/9dzzh5t_.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mo
17:17:43 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
Mean Absolute Percentage Error(MAPE)----------------------------------- 1.0568429806109756
{'changepoint_prior_scale': 0.1, 'holidays_prior_scale': 0.2, 'n_changepoints': 100, 'seasonality_mo
17:17:44 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/axqjuqac.json
Mean Absolute Percentage Error(MAPE)----------------------------------- 1.03165107620436
{'changepoint_prior_scale': 0.1, 'holidays_prior_scale': 0.2, 'n_changepoints': 100, 'seasonality_mo
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/afmet_m7.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mo
17:17:44 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
17:17:44 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/az8xshqm.json
Mean Absolute Percentage Error(MAPE)----------------------------------- 1.0490607232536082
{'changepoint_prior_scale': 0.1, 'holidays_prior_scale': 0.2, 'n_changepoints': 150, 'seasonality_mo
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/kc0gz7_k.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mo
17:17:45 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
17:17:45 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/i09yl8vv.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/_jsp_x5y.json
Mean Absolute Percentage Error(MAPE)----------------------------------- 1.0481192026094646
{'changepoint_prior_scale': 0.1, 'holidays_prior_scale': 0.2, 'n_changepoints': 150, 'seasonality_mo
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_m
```

```
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_percentage_error, mean_squared_error
from math import sqrt

p = d = q = range(0, 3)   # search space
results = []

for i in p:
    for j in d:
        for k in q:
            # try:
            order = (i, j, k)
            model = ARIMA(train_data_2['Total'], order=order)
            model_fit = model.fit()

            forecast = model_fit.forecast(steps=len(test_data_2))

            mape = mean_absolute_percentage_error(test_data_2['Total'], forecast)
            rmse = sqrt(mean_squared_error(test_data_2['Total'], forecast))
```

```
            results.append([order, model_fit.aic, mape, rmse])
            # except:
            #     continue

model_parameters = pd.DataFrame(results, columns=['Order', 'AIC', 'MAPE', 'RMSE'])
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-in
  warn('Non-invertible starting MA parameters found.'
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-in
  warn('Non-invertible starting MA parameters found.'
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-in
  warn('Non-invertible starting MA parameters found.'
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/sarimax.py:966: UserWarning: Non-st
  warn('Non-stationary starting autoregressive parameters'
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-in
  warn('Non-invertible starting MA parameters found.'
```

```
parameters = model_parameters.sort_values(by=['MAPE']).reset_index(drop=True)
print(parameters.head())
```

```
Empty DataFrame
Columns: [Order, AIC, MAPE, RMSE]
Index: []
```

```
if parameters.empty:
    print("No valid ARIMA models found. Try smaller (p,d,q) ranges or clean data.")
else:
    best_order = parameters.iloc[0]['Order']
    print("Best ARIMA order:", best_order)

    best_model = ARIMA(df_2['Total'], order=best_order).fit()
    print(best_model.summary())
```

```
No valid ARIMA models found. Try smaller (p,d,q) ranges or clean data.
```

```
model_parameters = pd.DataFrame(results, columns=['Parameters', 'AIC', 'MAPE', 'RMSE'])
```

```
# Taking best params
```

```
final_model = Prophet(holidays=holiday,
                      changepoint_prior_scale= 0.1,
                      holidays_prior_scale = 0.2,
                      n_changepoints = 100,
                      seasonality_mode = 'multiplicative',
                      weekly_seasonality=True,
                      daily_seasonality = True,
                      yearly_seasonality = True,
                      interval_width=0.95)
final_model.add_country_holidays(country_name='US')
final_model.fit(X_tr)
```

```
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/r0vm2im9.json
DEBUG:cmdstanpy:input tempfile: /tmp/tmpehwu9uox/4un_gamt.json
DEBUG:cmdstanpy:idx 0
DEBUG:cmdstanpy:running CmdStan, num_threads: None
DEBUG:cmdstanpy:CmdStan args: ['/usr/local/lib/python3.12/dist-packages/prophet/stan_model/prophet_mod
17:43:06 - cmdstanpy - INFO - Chain [1] start processing
INFO:cmdstanpy:Chain [1] start processing
17:43:07 - cmdstanpy - INFO - Chain [1] done processing
INFO:cmdstanpy:Chain [1] done processing
<prophet.forecaster.Prophet at 0x79c95dbf1610>
```

```
future = final_model.make_future_dataframe(periods=122, freq='D')
```

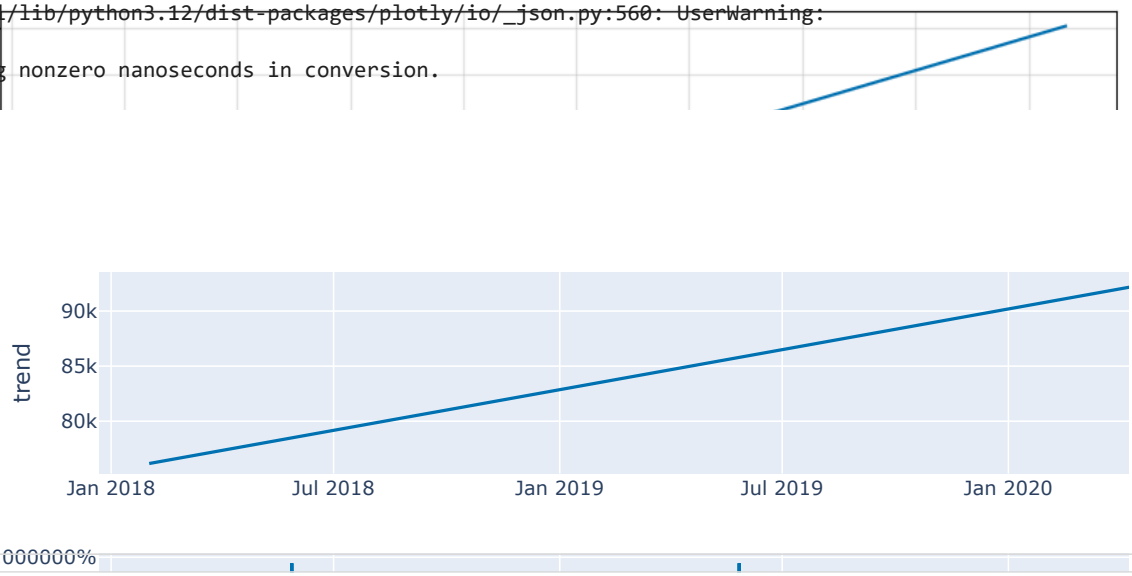
```
forecast = final_model.predict(future)
forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(7)
```

	ds	yhat	yhat_lower	yhat_upper
814	2020-04-25	11727.404577	-15385.861943	39459.949044
815	2020-04-26	-1573.550753	-30022.042647	28369.265429
816	2020-04-27	25615.198006	-2259.120642	53202.728007
817	2020-04-28	22843.821639	-4941.898421	48673.786470
818	2020-04-29	20609.757573	-6331.131931	49460.607327
819	2020-04-30	23300.399235	-3978.571584	52642.924982
820	2020-05-01	22562.749546	-4242.257245	52131.899245

```
fig =final_model.plot_components(forecast)
```

```
from prophet.plot import plot_plotly, plot_components_plotly
plot_components_plotly(final_model,forecast)
```

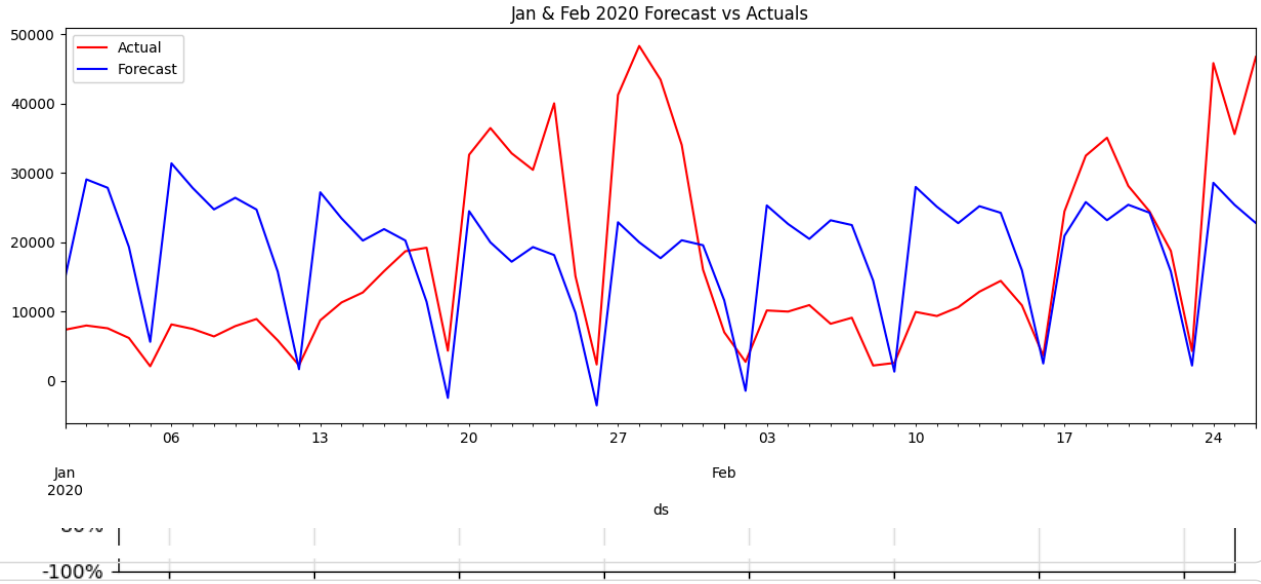
```
/usr/local/lib/python3.12/dist-packages/plotly/io/_json.py:560: UserWarning:

Discarding nonzero nanoseconds in conversion.
```




```
X_tst_final= final_model.predict(X_tst)
X_tst_final[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(7)
```

	ds	yhat	yhat_lower	yhat_upper
50	2020-02-20	25375.223840	-1783.715285	54620.624751
51	2020-02-21	24228.648139	-2934.169610	54483.247751
52	2020-02-22	15764.752328	-12684.911214	42752.922637
53	2020-02-23	2183.786836	-26577.659921	29947.893614
54	2020-02-24	28542.383326	578.709710	55516.976387
55	2020-02-25	25362.869175	-4282.115221	53281.649606
56	2020-02-26	22734.158146	-5686.413980	52132.579211


```
f, ax = plt.subplots(figsize=(14,5))
f.set_figheight(5)
f.set_figwidth(15)
X_tst.plot(kind='line',x='ds', y='y', color='red', label='Actual', ax=ax)
X_tst_final.plot(kind='line',x='ds',y='yhat', color='blue',label='Forecast', ax=ax)
plt.title('Jan & Feb 2020 Forecast vs Actuals')
plt.show()
```



```
MAPE = mean_absolute_percentage_error(X_tst['y'],abs(X_tst_final['yhat']))
print('MAPE', MAPE)
```

```
MAPE 1.03165107620436
```

```
X_tr
```

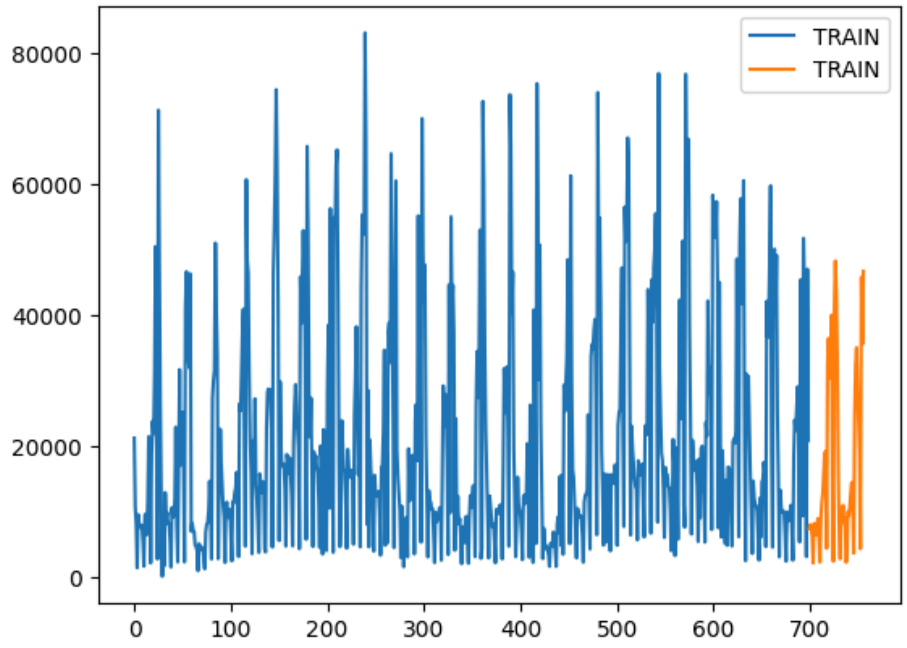
	ds	y
0	2018-02-01	21199.0
1	2018-02-02	10634.0
2	2018-02-03	7966.0
3	2018-02-04	1353.0
4	2018-02-05	9497.0
...	...	...
694	2019-12-27	44154.6
695	2019-12-28	21527.6
696	2019-12-29	3058.5
697	2019-12-30	47041.5
698	2019-12-31	20839.1

699 rows × 2 columns

Next steps: Generate code with X_tr | View recommended plots | New interactive sheet

```
X_tr['y'].plot(x='ds', legend=True, label='TRAIN')
X_tst['y'].plot(x='ds', legend=True, label='TRAIN')
```

```
<Axes: >
```




```
X_tr.set_index('ds').plot(legend=True,label='TRAIN')
X_tst.set_index('ds').plot(legend=True,label='TEST')
plt.show()
```